

Question Begets Question: Self-Evolving Curriculum for Reinforcement Fine-Tuning on Competition Mathematics

Longtian Bao^{1*}, Jianyou Wang^{2*}, Yang Zhang^{2*}, Youze Zheng^{2*}, Ramamohan Paturi²

¹University of Chicago

²University of California, San Diego

lobao@uchicago.edu, {jiw101, yaz124, yoz018, rpaturi}@ucsd.edu

Abstract

Teaching a language model a skill it has not mastered is obstructed by three recurring difficulties: training data is scarce, ground-truth reasoning traces are usually unavailable, and models often exhibit an apparent ceiling beyond which additional data yields no further improvement. We study these difficulties in a controlled setting, fine-tuning Qwen2.5-Math-7B on competition mathematics (AIME), a task on which it initially solves only 5.6% of problems (pass@1). To address data scarcity, we introduce Question-begets-Question (QbQ), a scalable procedure in which a teacher transforms existing problems into diverse variants that probe the same underlying skills; to model the absence of oracle reasoning, we train exclusively via reinforcement learning on problem statements and final answers, never on teacher reasoning traces. Static training on such data, however, plateaus well short of the task: real-plus-synthetic augmentation and non-curriculum QbQ generated synthetic data training cap pass@1 at 12.5% and 14.5% respectively, despite large increases in data. Our central finding is that this ceiling is not intrinsic to the model. We propose a self-evolving curriculum that, each round, evaluates the current checkpoint, seeds QbQ from the problems it can mostly get right, and trains on the resulting variants; under an identical data budget, this breaks the ceiling and lifts pass@1 to 16.5% with no sign of saturation after 20 rounds. Counterintuitively, we find that models improve when trained on variants of problems they can mostly get right, and that models trained this way go on to solve harder problems never seen during training.

1 Introduction

Teaching language models to acquire new skills is a fundamental problem in modern AI research. Although frontier large language models now match or exceed human experts in many domains, they continue to lag behind in others, such as constructing lengthy mathematical proofs (Petrov et al. 2025) or interpreting and litigating complex legal documents (Guha et al. 2023; Dahl et al. 2024). On tasks that challenge even human experts, including drug discovery (Guo and Ding 2026), clinical-trial outcome prediction (Wang et al. 2026), earthquake forecasting (Stockman, Lawson, and Werner 2026), and political forecasting (Karger et al. 2025), these models fare little better.

We attribute this gap to three recurring obstacles. First, for difficult tasks, training data is often scarce: M7+ earth-

quake catalogs, for instance, contain fewer than 2,000 usable records (Storchak et al. 2013). Second, even when data is abundant, correct reasoning traces are rarely available. Clinical-trial outcome prediction offers on the order of 500,000 labeled examples, yet none come with ground-truth explanations of why a trial succeeded or failed, and expert rationales in this setting are themselves often flawed and largely post hoc (Wang et al. 2026). Third, and most important, a model may face an apparent learning bottleneck, an ostensible ceiling on what it can master regardless of the amount of training data available (Yue et al. 2025).

We address each obstacle in turn. To overcome data scarcity, we present a method, Question-begets-Question (QbQ), that generates an abundance of synthetic data in a scalable manner while improving its diversity. To mirror settings in which oracle ground-truth reasoning traces are unavailable, we train our models on this synthetic data purely through reinforcement learning, never exposing them to reasoning traces. Finally, to test whether the ostensible ceiling can be broken under a fixed data budget, we propose an iterative, self-evolving curriculum that dynamically tailors training data to the evolving needs of the model, so that it may learn more effectively and surpass the ceiling.

While we lack the resources to study frontier models on the hardest open problems, we instead study a smaller model on a task that is genuinely hard for it. Concretely, we fine-tune Qwen2.5-Math-7B (released in 2024) on high-school competition mathematics (AIME). This is an appropriate testbed: the base model solves only about 5.6% of problems (pass@1) on the AIME 2025 and 2026 benchmarks, placing the task well beyond its current reach.

The natural remedies are only partly effective. Using the entire set of 1,005 problems with annotated solutions available prior to AIME 2024, supervised fine-tuning and GRPO raise pass@1 to 9.5% and 11.5%, respectively. Standard synthetic-data augmentation helps only modestly. When we use a stronger teacher, GPT-5-mini, to generate an additional 5,000 problems from the original 1,005, training on the combined 6,000 examples raises pass@1 to just 12.05% despite the sixfold increase in data. We partly attribute this limited benefit to a lack of diversity among the generated problems.

QbQ generates markedly more diverse synthetic data than standard augmentation, and training on 6,000 QbQ problems

*These authors contributed equally.

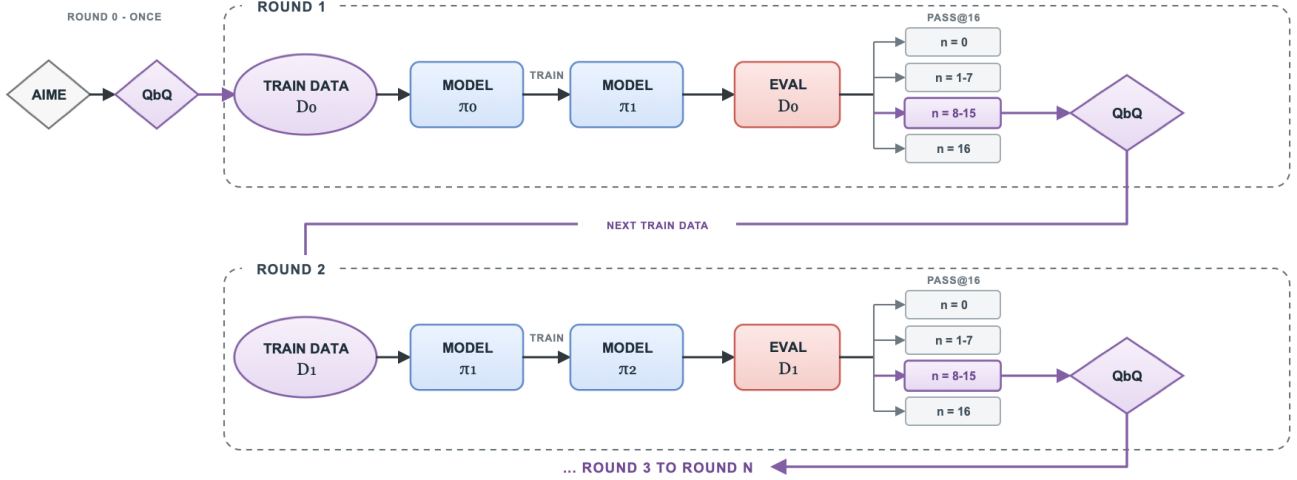


Figure 1: Overview of QbQ. The initial AIME training set is created once. In each subsequent round, the model is trained and evaluated, problems are grouped by pass@16, and only those with ($8 \leq n \leq 15$) are used to generate training data for the next round.

alone, with no real data, already improves performance to 14.5%. Yet the ostensible ceiling reappears Figure 2: over the final 30% of GRPO training, performance plateaus and does not improve as more data is consumed. Our central result is that this ceiling is not intrinsic to the model. When the same QbQ data is generated, organized, and trained upon within our self-evolving curriculum, and under the same data budget, the model breaks through to 16.5% pass@1, with no sign of saturation even after 20 rounds. The entire workflow is shown in Figure 1.

Our self-evolving curriculum runs for 20 rounds, each using GPT-5-mini to synthesize problems tailored to the current checkpoint. Each round has three stages. (1) *Self-evaluation*: the current checkpoint is evaluated on the previous round’s synthetic problems and sorts them into four bands, problems it has mastered (pass@16 = 16), problems it can mostly get right ($8 \leq \text{pass@16} \leq 15$), problems it sometimes gets right ($1 \leq \text{pass@16} \leq 7$), and problems it never gets right, which are too hard for it. (2) *Generation*: taking the problems it can mostly get right as seeds, the teacher synthesizes variants that probe a slightly different aspect of the same problem, reinforcing rather than escalating its understanding. (3) *Training*: we apply GRPO to the newly generated problems using only their questions and final answers, with no step-by-step solutions. Because the checkpoints never see or train on the teacher’s reasoning traces, this both models the no-oracle-reasoning setting and mitigates concerns about distillation.

The approach has two key strengths. Because each round’s problems are seeded from the previous round’s rather than from a fixed set, the diversity collapse that hampered static augmentation does not arise. And because problems are continually re-tailored to the current checkpoint, the model always trains on the most productive material, variants of problems it can mostly get right, which improves the effectiveness

of GRPO.

This last point underlies our central and counterintuitive finding. Whereas prior work argues that models improve by learning from their mistakes or hardest failures (Liang et al. 2025b; Chen et al. 2026a,b), we observe the opposite: a model improves most when trained on variants of problems it can mostly get right. Seeding from these near-successes at each round consistently outperforms seeding from the hardest problems. Perhaps most striking, a model trained under this curriculum goes on to solve harder problems it was never trained on. All experiments are conducted under matched compute and data budgets unless otherwise specified.

Contributions. Our contributions are as follows:

- We propose *Question-begets-Question* (QbQ), a scalable method for generating diverse synthetic training problems, and train on it purely via reinforcement learning without any teacher reasoning traces, modeling settings where ground-truth reasoning is unavailable.
- We document an ostensible performance ceiling under static training: both real-plus-synthetic augmentation and non-curriculum QbQ training plateau as data increases, capping pass@1 at 12.5% and 14.5% respectively.
- We introduce a self-evolving iterative curriculum that breaks this ceiling under an identical data budget, lifting Qwen2.5-Math-7B from 5.6% to 16.5% pass@1 with no sign of saturation after 20 rounds.
- We present a counterintuitive training principle: models improve most from variants of problems they can *mostly* get right, not from their hardest failures, and models trained this way generalize to harder problems never seen in training.

2 Related Work

Competition Mathematics Benchmarks: Mathematical benchmarks for language models span a wide range of difficulty. GSM8K and MATH-500 sit at the easier end (Cobbe et al. 2021; Hendrycks et al. 2021; Lightman et al. 2023), and current reasoning models perform well on both. At the hard end, Putnam, USAMO, and IMO problems typically demand written proofs, which are difficult to grade automatically (Tsoukalas et al. 2024; Art of Problem Solving 2024b; IMO Foundation 2024). Among competition benchmarks whose final answers are computation-centric, AIME is one of the hardest (Art of Problem Solving 2024a). FrontierMath reaches further still, into research mathematics (Glazer et al. 2025), but its problems were constructed to test AI systems rather than people. In this work, we focus on problems that have a life beyond AI evaluation. AIME problems are written for human competitors, every answer is an integer that a program can check, and four decades of contests supply more than a thousand problems in a single uniform format (§3.1). We therefore train and evaluate on AIME.

Curriculum Learning. Curriculum learning organizes training examples according to the learner’s current ability. In reinforcement fine tuning, this often means selecting problems that are neither already mastered nor completely beyond the model’s reach, since these two cases provide limited learning signal (Shi et al. 2026; Mahrooghi, Lotfi, and Abbe 2026). Recent work has explored adaptive task selection, problem reformulation, and training near the model’s current capability boundary (Chen et al. 2025, 2026b; Sundaram et al. 2026; Lee et al. 2026). Our method follows the same general idea but differs in how the curriculum is constructed. Rather than repeatedly sampling from a fixed collection, we generate new variants from problems that the model can solve often but not consistently. These variants become the basis for later rounds, allowing the training distribution to change as the model improves.

Synthetic Data and Self Evolving. Synthetic data is widely used to expand reasoning datasets when human written problems and solutions are limited. Early work generates mathematical training data through question rewriting, iterative question composition, and program based transformations (Yu et al. 2024; Liu et al. 2024; Khan et al. 2025). More recent methods place generation inside the training process, using the model’s weaknesses, current ability, or previous outputs to produce new problems for later training (Chen et al. 2025; Liang et al. 2025a,b; Zhang et al. 2026; Röpke et al. 2026; Huang et al. 2026a). At the same time, repeated training on model generated data can reduce diversity or cause performance to regress (Luo et al. 2026; Lin 2026). Our method addresses both concerns by generating variants from problems that match the model’s current ability and allowing successful variants to seed later rounds. This keeps the data learnable while changing its structure across rounds, so the training distribution develops together with the model rather than being generated once at the start.

Learning Ceilings in Reinforcement Fine Tuning. Prior work offers different explanations for why reinforcement

learning with verifiable rewards eventually stops improving. Some work connects this plateau to declining policy entropy and the resulting loss of exploration during training (Cui et al. 2025). A separate line of work focuses on the training data, showing that problems provide little useful signal once they become too easy, while problems that are too difficult rarely produce successful samples (Bae et al. 2026; Huang et al. 2026b). Recent results suggest that these limits can be overcome through longer training, better control of diversity, or curricula that introduce useful reasoning patterns near the model’s capability boundary (Liu et al. 2025; Yuan et al. 2026; Cai et al. 2026). Our work studies the role of the training distribution in this debate. Under the same data budget, static training reaches a plateau, while repeatedly generating variants from problems suited to the model’s current ability continues to improve performance.

3 Method

Our method couples two components. Question-begets-Question (QbQ) is a synthetic data generation procedure in which a teacher model turns an existing problem into new problems that probe a slightly different aspect of the same skills required to solve the existing problem. The self-evolving curriculum decides which problems QbQ expands and what the model trains on. Each of T rounds of the curriculum runs the following three stages: (1)Self-Evaluation: the current checkpoint is evaluated on the previous round’s training problems and categorizes them into four bands, the problems it already mastered, the problems it mostly get right, the problems it sometimes get right, and the problems it never solved; (2) QbQ Generation: the band of problems it can mostly get right become seeds, and QbQ expands exactly those into fresh variants; (3)Training: we apply GRPO on the new variants, using only their problem statements and answers generated by the teacher model. No step-by-step solution is used for training. This setup both mirrors settings in which oracle reasoning traces are unavailable and mitigates the concern that the gains are distilled from the teacher. Algorithm 1 details the QbQ self-evolving curriculum.

3.1 Setup and Notation

We formalize a problem to be a pair $p = (x_p, a_p)$: a problem statement x_p and an answer. A policy π maps x_p to a sampled solution y , and $\text{ans}(y)$ denotes the final answer.

For each problem p , to determine which difficulty band it belongs to with respect to the model checkpoint at round t , which is π_t , we draw $k = 16$ i.i.d. samples $y^{(1)}, \dots, y^{(k)} \sim \pi_t(\cdot \mid x_p)$ under fixed decoding parameters and count the number of correct final answers, denoted as n .

Self-evaluation sorts problems into four bands by this count: *mastered* ($n = 16$), problems the model can *mostly* get right ($8 \leq n \leq 15$), problems it *sometimes* gets right ($1 \leq n \leq 7$), and problems it *never* gets right ($n = 0$). The band where the seed questions are is denoted as the seed band $B = \{n : k/2 \leq n \leq k-1\}$, which means $B = \{8 \leq n \leq 15\}$.

For our QbQ synthetic data generation procedure, we use problems from the second band as seeds. Problems in this

Algorithm 1: The QbQ self-evolving curriculum

Require: pool $\mathcal{D}$; initial checkpoint π_0 ; rounds T ; rollout budget k ; seed band $B = \{n : k/2 \leq n \leq k-1\}$

- 1: $S_0 \leftarrow \{p \in \mathcal{D} : n_0(p) \in B\}$ $\triangleright$ mostly-right band of π_0 over the pool
- 2: **for** $t = 0, \dots, T-1$ **do**
- 3: $V_t \leftarrow \text{QBQ}(S_t) \triangleright$ synthesize and filter variants (§3.2)
- 4: score each $q \in V_t$: k rollouts of $\pi_t \Rightarrow n_t(q)$
- 5: $R_t \leftarrow \text{SELECTRL}(V_t)$
- 6: $\pi_{t+1} \leftarrow \text{GRPO}(\pi_t, R_t)$
- 7: re-score each $q \in R_t$: k rollouts of $\pi_{t+1} \Rightarrow n_{t+1}(q)$
- 8: $S_{t+1} \leftarrow \{q \in R_t : n_{t+1}(q) \in B\}$
- 9: **end for**
- 10: **return** π_T

band has the following characteristics: the model checkpoint π_t 's comprehension of these problems is incomplete, yet success is frequent enough that reinforcement learning algorithms have enough positive signals to learn from. Whereas for problems with $n = 0$ and problems with $n = 16$, both give every rollout in a group approximately the same reward and hence *zero* gradient (formalized in §3.3).

Generating questions from this band of ($8 \leq n \leq 15$) is a deliberate design choice, and our experiments will show that this design empirically outperforms the other design which is when we generate questions from the band ($1 \leq n \leq 7$). See Appendix A for details.

Train/Test Split The training pool $\mathcal{D}$ contains 1,005 AIME problems (1983–2024); the two most recent competition years are held out entirely for testing.

We use Qwen2.5-Math-7B for our main experiments, since it is released in 2024 and does not present any risk of contamination for our test set. We obtain the round-zero checkpoint π_0 by one supervised fine-tuning pass over teacher-written solutions to the pool via LoRA (Hu et al. 2021). This SFT step is done to ensure Qwen2.5-Math-7B understands the correct format for answering questions. Empirically the official solution is too abstract for Qwen2.5-Math-7B to understand, so we use GPT5-mini to elaborate on the solution. Other than the initialization step, GPT5-mini's reasoning traces or solutions are never used for training our model.

The initial set of problems is the mostly-right band of π_0 over the pool, which yields 133 seeds for our main backbone.

3.2 Question-begets-Question

At each round, QbQ transforms the current seeds S_t into a set of new training problems V_t (Algorithm 1, line 3). An unconstrained prompt such as “write a similar problem” tends to produce either superficial rewrites that preserve the underlying computation or unrelated problems that no longer test the target skill. The former yields near-duplicate training examples, whereas the latter introduces uncontrolled skill drift. To avoid these failure modes, QbQ conditions the teacher on one of five human-designed **structural transformation operators**. Each operator changes the problem structure while preserving its core skill, allowing the resulting variants to

probe complementary aspects of that skill without systematically increasing difficulty.

- **Generalize, then specialize.** Replace a fixed quantity in the seed with a parameter, then instantiate it in a new regime where the same lemma applies differently. For example, “trailing zeros of 100!” in base 10 becomes “trailing zeros of 100! in base 12,” which requires considering the exponents of both 2 and 3.
- **Parametrize and sum.** Introduce an index over configurations related to the seed and ask for an aggregate, with each term obtained using the same lemma. For example, “ordered sums of 4 positive integers equal to 20” becomes “ $\sum_{j=1}^5 f(j)$, where $f(j)$ counts ordered sums of j positive integers equal to 20.”
- **Change the queried quantity.** Retain the setup and givens but query a different quantity that still requires the seed's key lemma. For example, for a triangle with side lengths 13, 14, and 15, querying the inradius instead of the area still requires Heron's formula.
- **Inverse problem.** Fix the value of the original target and solve in the reverse direction for a parameter that attains it. For example, “how many positive solutions does $20m + 12n = 2012$ have?” becomes “find the least c for which $20m + 12n = c$ has exactly 5.”
- **Add a constraint layer.** Embed the seed computation within one additional routine reduction, without requiring a new theorem. For example, “ $7^{2024} \bmod 100$ ” becomes “ $(7^{2024} + 3^{2024}) \bmod 100$.”

We additionally require every variant to have an answer different from that of its parent. This inexpensive check rejects exact copies and answer-preserving surface rewrites, complementing the structural constraints imposed by the operators.

Planning and generation. The applicability of an operator depends on the seed. For example, a geometry problem may not admit a natural parametrization-and-summation transformation, while a fixed numerical computation may not have a meaningful inverse. For each seed, a planning call receives all five operator descriptions and selects the three most suitable operators. Invalid selections are replaced in a fixed order. This planning step encourages the variants of each seed to follow distinct structural directions.

Each selected (seed, operator) pair is then processed by a separate generation call. The teacher receives the seed, a reference solution, and the complete operator instruction. Changing only numerical values is explicitly disallowed, and the teacher may abstain if the operator is not applicable. Only the statement and answer are retained for training, consistent with the protocol in §3.3.

Diversity of the generated pool. We quantify problem diversity using the Vendi score, defined as the exponentiated entropy of the eigenspectrum of the pairwise cosine-similarity matrix over Qwen3-Embedding-8B representations. The score can be interpreted as the effective number of distinct problems in a pool. We compare QbQ synthetic data with standard synthetic data generated from real AIME

training questions. After normalization, we show that QbQ achieves a higher within-family Vendi score than standard synthetic data generation (1.59 vs. 1.25).

3.3 Self-Training

For each variant $q \in V_t$, we estimate its pre-update solve count $n_t(q)$, its number of correct answers among the k rollouts drawn during self-evaluation under π_t (Algorithm 1, line 4), following the definition in §3.1. These scores are used directly to construct the RL set, so selection requires no additional rollouts.

Selecting RL problems (SELECTRL). The GRPO learning signal for a problem is determined by reward variation within its sampled group. For a binary correctness reward with empirical solve rate $\hat{p} = n_t(q)/k$, the within-group standard deviation is $\sqrt{\hat{p}(1-\hat{p})}$. It is zero at $\hat{p} \in \{0, 1\}$ and maximized at $\hat{p} = \frac{1}{2}$, corresponding to $n_t(q) = k/2$ under our difficulty estimate. We therefore construct the RL set R_t of $N = 300$ problems from the eligible variants $\{q \in V_t : 1 \leq n_t(q) \leq k-1\}$ in two stages. First, to preserve parent coverage, we select for each seed its eligible variant whose solve count is closest to $k/2$. We then fill the remaining slots in increasing order of $|n_t(q) - k/2|$, subject to a per-seed cap.

GRPO. For each $q \in R_t$, we sample $m = 8$ completions $y_1, \dots, y_m \sim \pi_{\text{old}}(\cdot \mid x_q)$, where π_{old} is anchored at π_t at the start of the round, and compute a reward and group-normalized advantage for each completion:

$$\begin{aligned} r_i &= \mathbf{1}[\text{ans}(y_i) = a_q] + 0.1 b_i, \\ \hat{A}_i &= (r_i - \bar{r}) / s_r, \end{aligned} \quad (1)$$

where $b_i \in \{0, 1\}$ indicates compliance with the prescribed answer format, and the small bonus encourages well-formed completions. The quantities $\bar{r}$ and s_r denote the mean and standard deviation of $r_{1:m}$. Let $\rho_{i,\tau}(\theta)$ be the per-token importance ratio with respect to π_{old} . We maximize the clipped GRPO objective (Shao et al. 2024)

$$\begin{aligned} \bar{\rho}_{i,\tau} &= \text{clip}(\rho_{i,\tau}, 1-\varepsilon, 1+\varepsilon), \\ \ell_{i,\tau} &= \min(\rho_{i,\tau} \hat{A}_i, \bar{\rho}_{i,\tau} \hat{A}_i), \\ \mathcal{J}(\theta) &= \mathbb{E} \left[\frac{1}{m} \sum_{i=1}^m \frac{1}{|y_i|} \sum_{\tau} \ell_{i,\tau} \right], \end{aligned} \quad (2)$$

with clipping parameter $\varepsilon = 0.2$ and no KL penalty. We limit each round to 150 optimization steps, corresponding to one pass over R_t , and refresh the training set in every round. The resulting policy is denoted by π_{t+1} .

3.4 Evolving the Curriculum

Let $B = \{n : k/2 \leq n \leq k-1\}$ denote the *mostly-right* band from §3.1 (with $k = 16$, this is $8 \leq n \leq 15$). After the round- t update produces π_{t+1} , we re-evaluate the problems in R_t under π_{t+1} , obtaining updated solve counts $n_{t+1}(q)$, and retain those that fall in the seed band:

$$S_{t+1} = \{q \in R_t : n_{t+1}(q) \in B\}. \quad (3)$$

Here S_{t+1} is the seed set that QbQ expands in round $t+1$. Problems mastered after training, with $n_{t+1}(q) = k$, leave the curriculum. Problems that remain in B become seeds for the next round, while those below the band ($n_{t+1}(q) \leq k/2 - 1$) are discarded without replacement. Because R_t contains variants with solve counts from 1 to $k-1$, training can promote an initially below-band problem into B : such a problem enters the curriculum only once it becomes learnable for the current policy. The training distribution therefore evolves with model competence rather than following a pre-defined difficulty schedule. The number of rounds T is fixed in advance, and held-out performance is never used for selection or stopping. Beyond initialization, the only external supervision used by the loop consists of the pool and the synthetic problem answers.

4 Experiments

We study the three obstacles identified in Section 1 in a controlled setting where AIME is difficult for the base model and real training data are limited. Our experiments ask: (1) how far can standard training go on the available real problems; (2) does QbQ produce more useful synthetic data than static augmentation; and (3) with the QbQ data and training budget fixed, does organizing the data as a curriculum avoid the plateau of non-curriculum training?

4.1 Experimental Setup

Data and model. The real-data pool contains 1,005 AIME problems from 1983–2024, each paired with its official answer. We evaluate on all 60 problems from AIME 2025 and AIME 2026. These contests postdate the training pool and are not used for training, synthesis, curriculum construction, or model selection. The backbone is Qwen2.5-Math-7B with a 4,096-token context, and the teacher is gpt-5-mini-2025-08-07.

Initialization and baselines. We first train a supervised initialization, M0, on teacher-written solutions to the real-data pool. After filtering by answer format and length, 963 of the 1,005 traces remain. We train a rank-32 LoRA adapter for three epochs with scale 64, learning rate 10^{-5} , and a cosine schedule. Starting from M0, we apply GRPO for 500 updates on the 1,005 original AIME problems.

The static-augmentation baseline continues from the checkpoint after this GRPO stage. For each original AIME problem, the teacher is asked for five new problems using the same or closely related techniques. One pass over these variants requires 2,443 additional updates, or 2,943 updates including the first GRPO stage. This is 57 updates fewer than the 3,000-update budget used by the QbQ experiments.

QbQ comparisons. QbQ produces 300 problems in each of 20 rounds, for a total of 6,000. We compare two training strategies on exactly the same problem multiset:

- **QbQ non curriculum** globally shuffles the 6,000 problems and trains on them in one GRPO pass.
- **QbQ curriculum** retains the original sequence of twenty 300-problem batches and trains on them in that order.

Both strategies use the same initialization, optimizer, rollouts, and 3,000 GRPO updates. The controlled difference is therefore the curriculum organization, not the data, compute, or number of updates. All QbQ RL stages use only problem statements and final answers as supervision.

GRPO and evaluation. GRPO uses groups of eight completions, learning rate 3×10^{-6} , clipping parameter 0.2, and no KL penalty. The reward is exact answer correctness plus a 0.1 answer-format bonus. Training completions use temperature 1.0 and a 3,072-token limit; length-truncated completions are masked from the loss.

At evaluation time, we draw 16 completions for each held-out problem with temperature 0.7, top- $p = 0.8$, and a 3,072-token limit. Pass@1 is the fraction of correct answers across the resulting $60 \times 16 = 960$ completions. We evaluate every final checkpoint with three sampling seeds and report the mean and standard deviation.

4.2 Main Results

What each comparison tests. The first three rows measure how much can be learned from the original problem pool: supervised initialization is followed by GRPO on the same 1,005 problems. Static augmentation then asks whether a standard one-shot expansion of those problems moves performance beyond that point. QbQ non curriculum tests whether changing how the synthetic problems are generated produces a more useful training set. Finally, the comparison between the two QbQ rows is the strictest ablation in the table. It holds the 6,000 problems, initialization, optimizer, and number of updates fixed, and changes only whether the round order is retained.

Conventional training quickly exhausts the original problem pool. The base model reaches 5.62% pass@1, confirming that AIME is well beyond its initial capability. Supervised initialization improves pass@1 by 3.86 points to 9.48%, and GRPO on the original AIME problems reaches 11.53%. Further updates on these problems do not yield a stable gain: in a single-seed checkpoint sweep, pass@1 peaks at 13.65% after 200 updates but falls to 12.08% at update 500. The three-seed result at the final checkpoint is $11.53 \pm 0.39\%$. Because this run contains only 500 updates, it is a data-limited baseline rather than a compute-matched comparison with the QbQ runs.

More static data do not remove the plateau. Static augmentation increases the training pool by 5,000 usable variants and nearly matches the QbQ update budget, yet its final performance is $12.05 \pm 0.34\%$. This is only 0.52 points above GRPO on the original AIME problems. By contrast, QbQ non curriculum reaches $14.44 \pm 0.61\%$, improving by 2.39 points over static augmentation. Thus QbQ data are more useful than one-shot variants even without a curriculum, consistent with the Introduction’s data-diversity motivation.

The curriculum changes the learning trajectory. Figure 2 shows why endpoint accuracy alone understates the difference. QbQ non curriculum reaches 14.55% at 2,100 updates, then records 13.79%, 14.20%, and 13.96% at 2,400,

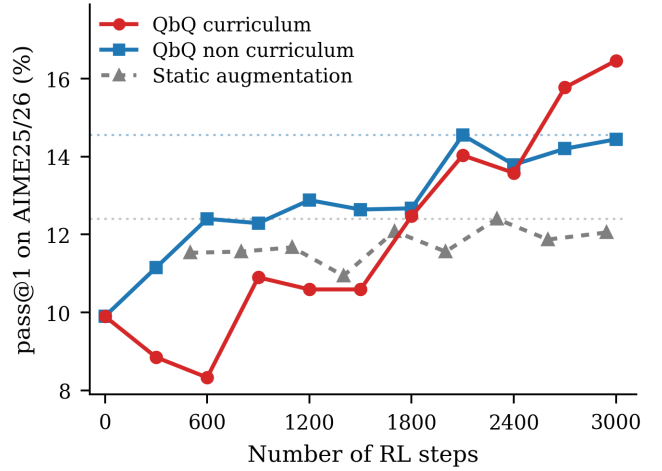


Figure 2: Pass@1 on AIME 2025 and 2026 during RL training. Static augmentation and QbQ non curriculum plateau, whereas QbQ curriculum continues to improve through 3,000 updates. Table 1 reports three-seed averages at the final checkpoints.

2,700, and 3,000 updates. The last 30% of training therefore fails to improve on the earlier checkpoint. Static augmentation is more limited still: its checkpoint evaluations never exceed 12.40% and show no sustained upward trend. Its three-seed endpoint in Table 1 is $12.05 \pm 0.34\%$. Over the same run, its training reward rises from approximately 0.28 to 0.54. The model is therefore fitting the fixed training pool more successfully without a corresponding gain on the held-out contests.

QbQ non curriculum exhibits the same problem at a higher accuracy. Each of its 6,000 problems is visited only once, so the plateau is not caused by training for multiple epochs on the same examples. The final 30% of training consumes 1,800 additional QbQ problems but does not exceed the result at 2,100 updates. Both fixed training distributions thus reach an observed ceiling within the available budget.

QbQ curriculum behaves differently. In the plotted trajectory, pass@1 rises from 13.58% at 2,400 updates to 15.77% at 2,700 and 17.08% at 3,000. It gains 3.50 points over the last 600 updates and finishes at its best plotted checkpoint, with no observed ceiling. The three-seed endpoint is $16.46 \pm 0.45\%$, 2.02 points above QbQ non curriculum.

Because the two QbQ strategies use the same examples and compute, this controlled comparison attributes the different trajectories to curriculum organization. The shape of the curves is consistent with the intended mechanism: global shuffling mixes problems produced for different stages of model competence, whereas the curriculum retains the round-by-round progression. Within the tested budget, this ordering separates the plateau from continued improvement.

Near-successes appear to provide better seeds than harder problems. We also vary the difficulty of the seed set. The main curriculum begins from 133 problems that M0 solves often but not reliably; a control run begins from 133 problems that M0 sometimes gets right. More details are

Model / Training Strategy	Seed 0	Seed 1	Seed 2	Mean $\pm \sigma$
Qwen2.5-Math-7B (base)	5.83	5.52	5.52	5.62 $\pm$ 0.15
M0 (supervised initialization)	9.90	9.48	9.06	9.48 $\pm$ 0.34
M0 + GRPO on original AIME problems	12.08	11.25	11.25	11.53 $\pm$ 0.39
+ Static augmentation	11.98	11.67	12.50	12.05 $\pm$ 0.34
QbQ without curriculum	15.31	14.06	13.96	14.44 $\pm$ 0.61
QbQ curriculum (ours)	16.04	16.25	17.08	16.46 $\pm$ 0.45

Table 1: Pass@1 (%) averaged over AIME 2025 and AIME 2026. Each row reports three evaluations of a fixed checkpoint. Both QbQ variants use the same 6,000 generated problems and 3,000 GRPO updates and differ only in whether the training problems are organized as a curriculum.

provided in Appendix A.

4.3 Transfer Beyond the Curriculum

We next test whether curriculum training transfers to problems that it does not select for RL. We define a fixed hard subset of 499 real problems on which M0 has zero sampled accuracy. These problems are outside the curriculum’s seed band and are never selected as seeds. After QbQ curriculum training, aggregate pass@1 on this subset rises from 0.0% to 5.3%.

5 Conclusion

We introduce Question-begets-Question (QbQ), a self-evolving curriculum that teaches a model a skill it has not mastered without any ground-truth reasoning traces. QbQ lifts Qwen2.5-Math-7B from 5.62% to 16.46% pass@1 on AIME 2025 and 2026 and keeps improving through 3,000 GRPO updates with no observed ceiling, while static augmentation and shuffled synthetic data both plateau under the same budget. Trained with our novel operator-based synthesis, QbQ generates markedly more diverse problems than one-shot augmentation (1.59 versus 1.25 Vendi score) and outperforms it by 4.41 points; holding the 6,000 problems, initialization, optimizer, and update count fixed and changing only the round ordering still yields 2.02 points, isolating curriculum organization as the source of the gain. By reseeding each round with problems that the current checkpoint solves often but not reliably, QbQ keeps rollout groups within GRPO’s high variance regime. It also outperforms seeding from the model’s hardest failures, achieving (16.46%) compared with (11.36%), contrary to the recommendation of mistake driven synthesis. The gains generalize beyond the curriculum’s own distribution: on 499 real problems the model never solves and the loop never selects as seeds, pass@1 rises from 0.0% to 5.3%. Because the policy trains only on problem statements and verified final answers, never on teacher reasoning, QbQ shows that an apparent learning ceiling may arise from the training distribution rather than the model itself. It also shows that this ceiling can be removed without distillation.

Acknowledgments

We gratefully acknowledge Professor Chenhao Tan and the University of Chicago Data Science Institute for providing access to the DSI computing cluster. The computational resources made available through the cluster were instrumental in supporting the experiments and analyses conducted in this work.

References

- Art of Problem Solving. 2024a. AIME Problems and Solutions. Art of Problem Solving Wiki, https://artofproblemsolving.com/wiki/index.php/AIME_Problems_and_Solutions. Accessed 2026-07-29.
- Art of Problem Solving. 2024b. USAMO Problems and Solutions. Art of Problem Solving Wiki, https://artofproblemsolving.com/wiki/index.php/USAMO_Problems_and_Solutions. Accessed 2026-07-29.
- Bae, S.; Hong, J.; Lee, M. Y.; Kim, H.; Nam, J.; and Kwak, D. 2026. Online Difficulty Filtering for Reasoning Oriented Reinforcement Learning. arXiv:2504.03380.
- Cai, P.; Fang, T.; Li, X.; Zeng, Q.; Li, G.; and Chen, J. 2026. Curriculum Reinforcement Learning Can Incentivize Reasoning Capacity in LLMs Beyond the Base Model. arXiv:2606.22317.
- Chen, J. C.-Y.; Peng, B. X.; Choubey, P. K.; Huang, K.-H.; Zhang, J.; Bansal, M.; and Wu, C.-S. 2026a. Nudging the Boundaries of LLM Reasoning. arXiv:2509.25666.
- Chen, J. C.-Y.; Prasad, A.; Khan, Z.; Singh, J.; Tian, R.; Stengel-Eskin, E.; and Bansal, M. 2026b. Cog-DRIFT: Exploration on Adaptively Reformulated Instances Enables Learning from Hard Reasoning Problems. arXiv:2604.04767.
- Chen, X.; Lu, J.; Kim, M.; Zhang, D.; Tang, J.; Piché, A.; Gontier, N.; Bengio, Y.; and Kamaloo, E. 2025. Self-Evolving Curriculum for LLM Reasoning. arXiv:2505.14970.
- Cobbe, K.; Kosaraju, V.; Bavarian, M.; Chen, M.; Jun, H.; Kaiser, L.; Plappert, M.; Tworek, J.; Hilton, J.; Nakano, R.; Hesse, C.; and Schulman, J. 2021. Training Verifiers to Solve Math Word Problems. arXiv:2110.14168.

- Cui, G.; Zhang, Y.; Chen, J.; Yuan, L.; Wang, Z.; Zuo, Y.; Li, H.; Fan, Y.; Chen, H.; Chen, W.; Liu, Z.; Peng, H.; Bai, L.; Ouyang, W.; Cheng, Y.; Zhou, B.; and Ding, N. 2025. The Entropy Mechanism of Reinforcement Learning for Reasoning Language Models. arXiv:2505.22617.
- Dahl, M.; Magesh, V.; Suzgun, M.; and Ho, D. E. 2024. Large Legal Fictions: Profiling Legal Hallucinations in Large Language Models. arXiv:2401.01301.
- Glazer, E.; Erdil, E.; Besiroglu, T.; Chicharro, D.; Chen, E.; Gunning, A.; Olsson, C. F.; Denain, J.-S.; Ho, A.; de Oliveira Santos, E.; Järvinen, O.; Barnett, M.; Sandler, R.; Vrzsala, M.; Sevilla, J.; Ren, Q.; Pratt, E.; Levine, L.; Barkley, G.; Stewart, N.; Grechuk, B.; Grechuk, T.; Enu-gandla, S. V.; and Wildon, M. 2025. FrontierMath: A Benchmark for Evaluating Advanced Mathematical Reasoning in AI. arXiv:2411.04872.
- Guha, N.; Nyarko, J.; Ho, D. E.; Ré, C.; Chilton, A.; Narayana, A.; Chohlas-Wood, A.; Peters, A.; Waldon, B.; Rockmore, D. N.; Zambrano, D.; Talisman, D.; Hoque, E.; Surani, F.; Fagan, F.; Sarfaty, G.; Dickinson, G. M.; Por-rat, H.; Hegland, J.; Wu, J.; Nudell, J.; Niklaus, J.; Nay, J.; Choi, J. H.; Tobia, K.; Hagan, M.; Ma, M.; Livermore, M.; Rasumov-Rahe, N.; Holzenberger, N.; Kolt, N.; Henderson, P.; Rehaag, S.; Goel, S.; Gao, S.; Williams, S.; Gandhi, S.; Zur, T.; Iyer, V.; and Li, Z. 2023. LegalBench: A Collaboratively Built Benchmark for Measuring Legal Reasoning in Large Language Models. arXiv:2308.11462.
- Guo, J.; and Ding, S. 2026. Do Larger Models Really Win in Drug Discovery? A Benchmark Assessment of Model Scaling in AI-Driven Molecular Property and Activity Prediction. arXiv:2604.26498.
- Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. arXiv:2103.03874.
- Hu, E. J.; Shen, Y.; Wallis, P.; Allen-Zhu, Z.; Li, Y.; Wang, S.; Wang, L.; and Chen, W. 2021. LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.
- Huang, C.; Yu, W.; Wang, X.; Zhang, H.; Li, Z.; Li, R.; Huang, J.; Mi, H.; and Yu, D. 2026a. R-Zero: Self-Evolving Reasoning LLM from Zero Data. arXiv:2508.05004.
- Huang, Y.; Wen, Z.; Chi, Y.; Wei, Y.; Singh, A.; Liang, Y.; and Chen, Y. 2026b. On the Emergence of Implicit Curriculum in RLVR Learning Dynamics. arXiv:2602.14872.
- IMO Foundation. 2024. International Mathematical Olympiad. <https://www.imo-official.org/>. Accessed 2026-07-29.
- Karger, E.; Bastani, H.; Yueh-Han, C.; Jacobs, Z.; Hal-lawi, D.; Zhang, F.; and Tetlock, P. E. 2025. Forecast-Bench: A Dynamic Benchmark of AI Forecasting Capabilities. arXiv:2409.19839.
- Khan, Z.; Stengel-Eskin, E.; Prasad, A.; Cho, J.; and Bansal, M. 2025. Executable Functional Abstractions: Inferring Generative Programs for Advanced Math Problems. arXiv:2504.09763.
- Lee, B.-K.; Lu, X.; Diao, S.; Kang, M.; Muralidharan, S.; Sapra, K.; Tao, A.; Molchanov, P.; Choi, Y.; Wang, Y.-C. F.; and Hachiuma, R. 2026. Zone of Proximal Policy Optimization: Teacher in Prompts, Not Gradients. arXiv:2606.18216.
- Liang, X.; Li, Z.; Gong, Y.; Shen, Y.; Wu, Y. N.; Guo, Z.; and Chen, W. 2025a. Beyond Pass@1: Self-Play with Variational Problem Synthesis Sustains RLVR. arXiv:2508.14029.
- Liang, X.; Li, Z.-Z.; Gong, Y.; Wang, Y.; Zhang, H.; Shen, Y.; Wu, Y. N.; and Chen, W. 2025b. SwS: Self-aware Weakness-driven Problem Synthesis in Reinforcement Learning for LLM Reasoning. arXiv:2506.08989.
- Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let's Verify Step by Step. arXiv:2305.20050.
- Lin, J. 2026. Self-Improvement Can Self-Regress: The Rise-and-Collapse Failure Mode of LLM Self-Training. arXiv:2606.21090.
- Liu, H.; Zhang, Y.; Luo, Y.; and Yao, A. C.-C. 2024. Augmenting Math Word Problems via Iterative Question Composing. arXiv:2401.09003.
- Liu, M.; Diao, S.; Lu, X.; Hu, J.; Dong, X.; Choi, Y.; Kautz, J.; and Dong, Y. 2025. ProRL: Prolonged Reinforcement Learning Expands Reasoning Boundaries in Large Language Models. arXiv:2505.24864.
- Luo, X.; Huang, Y.; Guo, K.; He, P.; Zou, C.; Hua, T.; and Zhang, X. 2026. Learning from Synthetic Data without Model Collapse in Iterative Instruction Tuning. arXiv:2607.17043.
- Mahrooghi, I.; Lotfi, A.; and Abbe, E. 2026. Goldilocks RL: Tuning Task Difficulty to Escape Sparse Rewards for Reasoning. arXiv:2602.14868.
- Petrov, I.; Dekoninck, J.; Baltadzhiev, L.; Drencheva, M.; Minchev, K.; Balunović, M.; Jovanović, N.; and Vechev, M. 2025. Proof or Bluff? Evaluating LLMs on 2025 USA Math Olympiad. arXiv:2503.21934.
- Röpke, W.; Coward, S.; Lupu, A.; Foster, T.; Rocktäschel, T.; and Foerster, J. 2026. DéjàQ: Open-Ended Evolution of Diverse, Learnable and Verifiable Problems. arXiv:2601.01931.
- Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. arXiv:2402.03300.
- Shi, T.; Wu, Y.; Song, L.; Zhou, T.; and Zhao, J. 2026. Efficient Reinforcement Finetuning via Adaptive Curriculum Learning. arXiv:2504.05520.
- Stockman, S.; Lawson, D.; and Werner, M. 2026. EarthquakeNPP: A Benchmark for Earthquake Forecasting with Neural Point Processes. arXiv:2410.08226.
- Storchak, D. A.; Di Giacomo, D.; Bondár, I.; Engdahl, E. R.; Harris, J.; Lee, W. H. K.; Villaseñor, A.; and Bormann, P. 2013. Public Release of the ISC-GEM Global Instrumental Earthquake Catalogue (1900–2009). *Seismological Research Letters*, 84(5): 810–815.
- Sundaram, S.; Quan, J.; Kwiatkowski, A.; Ahuja, K.; Olivier, Y.; and Kempe, J. 2026. Teaching Models to Teach Themselves: Reasoning at the Edge of Learnability. arXiv:2601.18778.

Tsoukalas, G.; Lee, J.; Jennings, J.; Xin, J.; Ding, M.; Jennings, M.; Thakur, A.; and Chaudhuri, S. 2024. Putnam-Bench: Evaluating Neural Theorem-Provers on the Putnam Mathematical Competition. arXiv:2407.11214.

Wang, J.; Zheng, Y.; Bao, L.; Zhang, H.; Zheng, Q.; Chen, Y.; Zhang, Y.; Feng, M.; Khan, M.; Sehgal, A. K.; Rosin, C. D.; Paturi, R.; Dube, U.; and Bergen, L. 2026. CT Open: An Open-Access, Uncontaminated, Live Platform for the Open Challenge of Clinical Trial Outcome Prediction. arXiv:2604.16742.

Yu, L.; Jiang, W.; Shi, H.; Yu, J.; Liu, Z.; Zhang, Y.; Kwok, J. T.; Li, Z.; Weller, A.; and Liu, W. 2024. MetaMath: Bootstrap Your Own Mathematical Questions for Large Language Models. arXiv:2309.12284.

Yuan, S.; Chen, J.; Zheng, J.; Li, M.; Feng, L.; Wang, D.; Xiang, T.; Liu, T.; and An, B. 2026. Understanding Diversity Collapse in RLVR via the Lens of Overtraining. arXiv:2606.15455.

Yue, Y.; Chen, Z.; Lu, R.; Zhao, A.; Wang, Z.; Yue, Y.; Song, S.; and Huang, G. 2025. Does Reinforcement Learning Really Incentivize Reasoning Capacity in LLMs Beyond the Base Model? arXiv:2504.13837.

Zhang, R.; Li, R.; Ma, Z.; Qiu, W.; Tao, C.; Wang, Y.; and Chu, X. 2026. D²Evo: Dual Difficulty-Aware Self-Evolution for Data-Efficient Reinforcement Learning. arXiv:2605.17037.

A Additional Experiments

Our proposed curriculum deliberately generates new problems from the mostly-right band, $8 \leq n \leq 15$, which contains problems the model solves often but not yet reliably (§3.2). We compare it with two intuitive alternatives. The harder-seed curriculum uses seeds with solve counts $1 \leq n \leq 7$, whereas the mixed-seed curriculum uses seeds with solve counts $1 \leq n \leq 15$, where n denotes the number of correct completions among 16 rollouts. We rerun the complete self-evolving loop with each seed band. Neither alternative reproduces the gain of the mostly-right curriculum.

Setup. The comparison uses the same initialization, teacher, operator library, generation framework, and held-out evaluation as the main run (Table 3). Each arm is configured for 20 rounds, with 300 RL problems and 150 optimizer steps per round. The arms use the following seed bands:

- **Harder seeds** ($1 \leq n \leq 7$). The initial set contains 133 problems sampled from the 362 problems that M0 solves between 1 and 7 times in 16 attempts. This matches the size of the main run’s initial seed set.
- **Mixed seeds** ($1 \leq n \leq 15$). The initial set contains 133 problems sampled from the union of the two nondegenerate bands (495 problems). Stratification by solve count gives 96 sometimes-right and 37 mostly-right seeds, approximately matching the composition of the full pool.

Training condition	$s = 0$	$s = 1$	$s = 2$	Mean $\pm \sigma$
Original-problem GRPO	12.08	11.25	11.25	11.53 $\pm$ 0.39
Mostly right (8–15)	16.04	16.25	17.08	16.46 $\pm$ 0.45
Harder (1–7)	11.67	11.15	11.25	11.35 $\pm$ 0.23
Mixed (1–15)	12.29	11.88	11.56	11.91 $\pm$ 0.30

Table 2: Endpoint Pass@1 (%) on AIME 2025 and AIME 2026. Each entry averages the two contest years over three evaluation seeds. The original-problem GRPO result is reproduced from Table 1 as a reference; the remaining rows compare curricula defined by different seed bands. σ denotes the population standard deviation across evaluation seeds.

Results. Table 2 and Figure 3 show the outcome. The harder- and mixed-seed curricula finish at $11.35 \pm 0.23\%$ and $11.91 \pm 0.30\%$, respectively. Both endpoints lie within 0.4 percentage points of the original-problem GRPO baseline and fail to reproduce the gain of the mostly-right curriculum, which reaches $16.46 \pm 0.45\%$. The seed band is consequently not a minor tuning detail: it separates a curriculum that works from curricula that contribute no material gain.

The two alternatives fail in different ways. The harder-seed curriculum initially outpaces the main run and reaches 14.17% halfway through training, but then loses the entire gain. The mixed curriculum peaks early at 13.33% and subsequently fluctuates between approximately 10% and 12.2%, never escaping its initial plateau. The mostly-right curriculum follows the opposite trajectory: it trails both alternatives early, improves sharply late in training, and finishes at its maximum observed value.

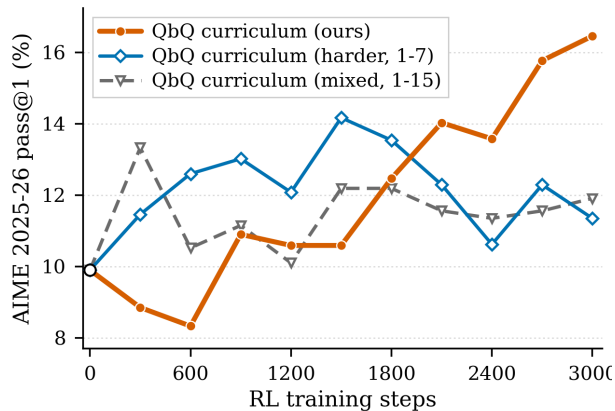


Figure 3: Held-out Pass@1 during training for the three seed bands. Intermediate points use one evaluation seed; each endpoint is the three-seed mean in Table 2. The mostly-right curriculum is the only arm that continues improving through the end of training. The harder-seed curriculum peaks mid-way and then declines, whereas the mixed curriculum never escapes its early plateau.

Why harder seeds fail. A harder seed does not necessarily provide a stronger learning signal. Its variants are more likely to remain beyond the model’s reliable solving range, producing rollout groups with no correctness variation. When the format reward is also constant, their standardized advantages vanish; even otherwise, they provide no relative correctness signal. By contrast, variants of mostly-right parents tend to lie near the model’s competence boundary, where groups contain both correct and incorrect completions and yield informative relative advantages. The harder majority therefore shifts the mixed curriculum toward lower-signal groups.

These results support the paper’s central causal claim: progress comes from generating just beyond the model’s current competence boundary, not from maximizing seed difficulty. Problems that are already solved often but not reliably provide the productive frontier for self-evolution; shifting the curriculum toward harder seeds removes that advantage.

B The Operator Details

This section provides the complete definitions and worked examples for the five structural transformation operators introduced in §3.2. Each operator changes the problem structure while preserving its core mathematical skill. The examples are deliberately elementary so that the structural transformation is transparent; they are illustrative rather than training instances. Every generated variant is additionally required to have an answer different from that of its parent.

B.1 Operator Definitions and Worked Examples

O1: Generalize, then specialize. Identify a value that the seed holds fixed, promote it to a free parameter, and specialize the parameter to a different concrete case in which the

same lemma applies in a new regime. Merely increasing a numerical value does not qualify.

Seed. Find the number of trailing zeros of $100!$ when written in base 10.

Variant. Find the number of trailing zeros of $100!$ when written in base 12.

Analysis. Both problems use Legendre’s formula for prime exponents in a factorial. The seed fixes the base as $10 = 2 \cdot 5$ and reduces to counting factors of 5. The variant uses $12 = 2^2 \cdot 3$, so the solver must compare the exponents of 2 and 3. The lemma is unchanged, but it is applied in a structurally different regime.

O2: Parametrize and sum. Starting from a seed that concerns one configuration, introduce an index over a small family of related configurations and ask for an aggregate. Each term must be obtained using the seed’s lemma; the aggregation supplies the new structure.

Seed. In how many ways can 20 be written as an ordered sum of 4 positive integers?

Variant. For $j = 1, \dots, 3$, let $f(j)$ be the number of ways to write 20 as an ordered sum of j positive integers. Find $\sum_{j=1}^3 f(j)$.

Analysis. Each term is a composition count, $f(j) = \binom{19}{j-1}$, obtained by the same stars-and-bars argument as the seed. The variant treats this count as a function of j and aggregates its values over a family of configurations, giving $1 + 19 + 171 = 191$.

O3: Change the queried quantity. Retain the setup, given values, and governing relations, but ask for a different unknown that still requires the seed’s key lemma.

Seed. A triangle has side lengths 13, 14, and 15. Find its area.

Variant. A triangle has side lengths 13, 14, and 15. Find its inradius.

Analysis. Both solutions require Heron’s formula, which gives area 84. The seed stops at the area, whereas the variant uses the additional routine relation $r = K/s = 84/21 = 4$. Thus, the seed’s key lemma remains necessary.

O4: Inverse problem. When the seed supplies parameters and asks for a resulting value, instead fix a target value of that result and solve in reverse for a parameter that attains it. The governing relation remains the same.

Seed. How many ordered pairs of positive integers (m, n) satisfy $20m + 12n = 2012$?

Variant. Find the smallest positive integer c for which $20m + 12n = c$ has exactly 5 solutions in positive integers (m, n) .

Analysis. Both problems count positive solutions of a linear Diophantine equation. The seed computes the count for a fixed right-hand side, whereas the variant fixes the count and solves for the right-hand side.

<i>Loop constants</i>	
backbone / context length	Qwen2.5-Math-7B / 4,096
rollouts per difficulty measurement k	16
seed band B	$8 \leq n \leq 15$
variants per seed / maximum waves	3 / 3
RL-set size N	300
rounds T (fixed in advance)	20
GRPO steps per round / total	150 / 3,000
<i>Difficulty measurement</i>	
temperature / top- p / top- k	0.7 / 0.8 / 20
repetition penalty / token limit	1.05 / 3,072
<i>Initialization of π_0</i>	
data	963 teacher solutions
adapter	LoRA $r = 32$, $\alpha = 64$
dropout / epochs	0.05 / 3
schedule	lr 10^{-5} , cosine
warmup ratio	0.03
<i>Per-round GRPO</i>	
optimizer / learning rate	AdamW / 3×10^{-6}
momenta	$\beta_1=0.9$, $\beta_2=0.99$
gradient-norm clip	0.2
group size m / clip ε / KL	8 / 0.2 / 0
reward	correctness + 0.1·format
sampling temperature / token limit	1.0 / 3,072
optimizer steps / total completions per step	150 / 16
<i>Hold-out evaluation</i>	
completions per problem	16
temperature / top- p / top- k	0.7 / 0.8 / disabled
repetition penalty / token limit	1.0 / 3,072
sampling seeds	3 (final), 1 (curves)

Table 3: Main experimental hyperparameters. LoRA follows Hu et al. (2021). The backbone is Qwen2.5-Math-7B; the loop itself is backbone-agnostic.

O5: Add a constraint layer. Embed the seed’s core computation in a slightly larger setup that requires one additional routine reduction back to the core. The added layer must not require a new theorem.

Seed. Find the remainder when 7^{2024} is divided by 100.

Variant. Let $N = 7^{2024} + 3^{2024}$. Find the remainder when N is divided by 100.

Analysis. The primary skill, reducing large powers modulo 100 via periodicity, is unchanged. The variant adds one application of the same routine to 3^{2024} , followed by modular addition.

C Hyperparameters and Protocols

Table 3 summarizes the main-loop and evaluation settings. Baseline-specific training settings are described in the main text.

Prompting and grading. All policy sampling uses the backbone’s chat template with the instruction “Please reason

step by step, and put your final answer within `\boxed{\}`” appended to the problem statement. The last boxed integer is normalized and compared with the reference answer. A completion with no extractable boxed integer is counted as incorrect.

Supervised initialization. The initial policy π_0 is trained once, before the iterative loop, on one teacher-written solution per real-pool problem. A LoRA adapter is trained on all linear layers with rank 32, scale 64, and dropout 0.05 for three epochs. We use a learning rate of 10^{-5} , a cosine schedule, and a warmup ratio of 0.03, and then merge the adapter into the backbone. This is the only stage that trains on solution text; subsequent stages use problem statements and final answers only (§3.3).

Synthesis. The teacher is gpt-5-mini-2025-08-07. Each round targets three variants per seed. A planning call selects three of the five operators, and one generation call is issued for each selected operator.

Difficulty measurement. The solve count $n_t(p)$ is the number of correct answers among $k = 16$ rollouts sampled with temperature 0.7, top- p 0.8, top- k 20, repetition penalty 1.05, and a 3,072-token limit. Because this count is a sum of sixteen Bernoulli outcomes, re-measurement can move problems near a band boundary. Each selection therefore uses the count available at selection time; later re-scoring produces a new count only for subsequent decisions.

C.1 RL Problem-Set Construction

Algorithm 2 gives the main-run procedure summarized in §3.3. First, every seed with an eligible variant retains the variant whose solve count is closest to $k/2$. This coverage step prevents lineages from disappearing solely because other seeds have more variants. The remaining slots are filled by the same distance criterion, subject to a per-seed cap. Since $\sqrt{\hat{p}(1-\hat{p})}$ is maximized at $\hat{p} = 1/2$, this ordering prioritizes problems expected to provide the strongest within-group reward variation.

Variants with counts $n \in \{0, k\}$ are excluded from the primary pool because their rollout groups have constant correctness reward at the selection snapshot. They are used only as a last-resort top-up when the eligible pool is too small. The main 20-round run always had more than N eligible variants, and the degenerate top-up was never used.

C.2 GRPO Settings

In the main run, each round performs 150 optimizer steps, corresponding to one pass over $N = 300$ selected problems at two problems per step. For each problem, the policy samples $m = 8$ completions at temperature 1.0 with a 3,072-token limit; length-truncated completions are masked from the loss. The reward in Eq. (1) is answer correctness plus a 0.1 well-formed-answer bonus and is standardized within each group. We use AdamW with a constant learning rate of 3×10^{-6} , no warmup, $\beta_1 = 0.9$, $\beta_2 = 0.99$, gradient-norm clipping at 0.2, GRPO clipping parameter $\varepsilon = 0.2$, and no KL penalty. Optimizer state is re-initialized at the beginning of each round, and training uses bfloat16.

Algorithm 2: SelectRL (one round; $N = 300$, $k = 16$)

```

1:  $E \leftarrow \{q \in V_t : 1 \leq n_t(q) \leq k - 1\}$ 
2:  $d(q) \leftarrow |n_t(q) - k/2|$ 
3: if  $|E| \leq N$  then
4:    $R \leftarrow E$ 
5: else
6:    $R \leftarrow \emptyset$ 
7:   for each seed  $s$  represented in  $E$  do
8:     add to  $R$  the variant of  $s$  minimizing  $d(q)$ 
9:   end for
10:   $\text{cap} \leftarrow \lceil N / \# \text{covered seeds} \rceil$ 
11:  for  $q \in E \setminus R$  in increasing  $d(q)$  do
12:    add  $q$  if its seed has fewer than cap representatives;
    stop at  $|R| = N$ 
13:  end for
14:  if  $|R| < N$  then
15:    relax the cap and fill from  $E \setminus R$  in the same order
16:  end if
17: end if
18: if  $|R| < N$  then
19:   top up from  $n_t = 0$ , then  $n_t = k$ 
20: end if
21: return  $R$ 
```

C.3 Computing Infrastructure

All experiments ran on a single Linux server with NVIDIA H100 80GB GPUs; each training or evaluation job uses between two and eight GPUs, with four the typical allocation. The software stack is PyTorch 2.10 with CUDA 12.8, TRL 1.4.0 for GRPO with vLLM 0.19.1 generating rollouts in colocated mode, FSDP2 sharding, and Transformers 5.8.1; exact pinned versions of every package ship with the code appendix. One curriculum round (synthesis, 150 GRPO steps, re-scoring, and held-out evaluation) takes roughly 90 minutes on four GPUs, so a full 20-round run completes in about 30 hours; teacher API usage is approximately \$7 per round.

C.4 Held-Out Evaluation

Held-out accuracy is measured on all 60 problems from AIME 2025 and AIME 2026. We draw $k = 16$ completions per problem with temperature 0.7, top- p 0.8, top- k disabled, repetition penalty 1.0, and a 3,072-token limit. Pass@1 is the fraction of correct answers among the resulting 960 completions. Each final checkpoint in the main results table is evaluated with three sampling seeds, and the mean and standard deviation are reported. Per-round curves use one fixed sampling seed.